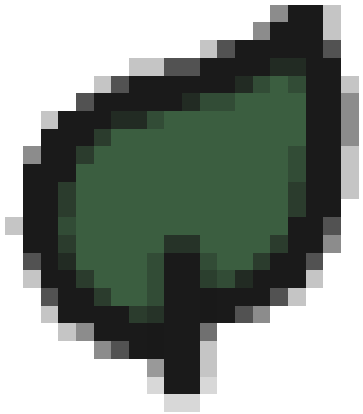


Dryad Language Specification

Pedro Kaleb

Edition 1.0 — Draft

October 2026

Dryad Language Project

This document specifies the Dryad programming language. It is the normative reference for language implementors. Clauses are normative unless explicitly marked informative; proposals and implementation guidance are confined to the annexes.

Licensed under the MIT License.

Abstract

This document specifies the Dryad programming language: its lexical grammar, type system, expressions, statements, declarations, module system, execution model, error model, concurrency primitives, foreign function interface, runtime architecture, standard library, bytecode instruction set, and ahead-of-time compilation pipeline. It is the normative reference for implementors of Dryad and supersedes the Dryad Conventions 1.0 draft as the authoritative specification.

Contents

Status of This Document **2**

Introduction **3**
Design goals . . . 3
Reading guide . . . 4

1 Scope **4**

2 Conformance **5**

3 Normative References **6**

4 Terms and Definitions **8**

5 Notational Conventions **10**
5.1 Normative keywords . . . 10
5.2 Grammar notation . . . 10
5.3 Algorithm conventions . . . 11
5.4 Completion records . . . 12
5.5 Type notation . . . 12
5.6 Mathematical notation . . . 13
5.7 Normative and informative marking . . . 13
5.8 Document versioning . . . 14

6 Overview **14**
6.1 Language identity . . . 14
6.2 Design axioms . . . 14
6.3 Surface language . . . 14
6.4 Execution architecture . . . 15
6.5 Native boundary and self-hosting . . . 15
6.6 Document organisation . . . 16

Status of This Document

Document identifier. DRYAD-SPEC-1.0-DRAFT.

Status. Draft Standard. This edition is complete for review but is not yet a ratified standard. Implementations claiming conformance to this edition MUST identify the edition and the conformance class they implement (Clause 2).

Normative content. Clauses 1 through 20 and Annexes A through D are normative. The Introduction, Clause 21, Clause 24, and Annexes E through G are informative. A clause or annex is normative unless it

is explicitly marked *informative*. Within normative clauses, text marked **NOTE** is informative.

Relationship to prior documents. This document consolidates and replaces the content of *Dryad Conventions 1.0* (the `conventions` publication). It also supersedes *Dryad Theoretical Foundation v2.0* as the authoritative language specification; the v2 document is retained as an informative architecture paper. Where this document and any prior document disagree, this document prevails.

Editorial decisions. The conventions draft contains internal inconsistencies. Every deliberate resolution is recorded in Annex E. An implementation that follows Annex E is following this specification, not deviating from it.

Change control. Changes to normative content require a new edition or a published erratum. Editorial changes that do not alter normative behaviour may be published as a revision of the same edition.

Licence. This specification is published under the MIT Licence.

Introduction

This Introduction is informative.

Dryad is a multi-paradigm programming language designed for clarity, portability, and deterministic execution. Its syntax is deliberately close to JavaScript and TypeScript, while its runtime architecture borrows the micro-kernel discipline of systems languages: a small native core exposes a fixed set of intrinsics, and the entire standard library is written in Dryad itself.

The language is defined by the interaction of four layers:

1. **Source language.** A statically scoped, dynamically typed language with optional strict typing.
2. **Execution tiers.** A tree-walking interpreter, a stack-based bytecode virtual machine, and an ahead-of-time compiler. The tiers are required to be observationally equivalent.
3. **Native boundary.** A minimal, explicitly specified intrinsic interface through which all operating-system interaction occurs.
4. **Standard library.** Pure Dryad modules that define the user-visible platform, including I/O, networking, collections, concurrency, and diagnostics.

This specification is written for implementors. It does not teach the language, and it does not describe the current implementation. It defines what a conforming implementation MUST do. Where the current implementation differs, the implementation is non-conforming; Clause 24 and Annex E identify known gaps for planning purposes.

Design goals

Goal	Normative consequence
Syntactic familiarity	The surface grammar follows JavaScript/TypeScript conventions unless this document states otherwise.
Paradigm flexibility	Procedural, object-oriented, and functional constructs are first-class and freely composable.
Optional typing	Dynamic typing is the default; strict typing is opt-in per module, function, or declaration.
Triple execution	The interpreter, bytecode VM, and AOT compiler MUST preserve the same observable semantics.

Goal	Normative consequence
Native concurrency	Threads, mutexes, and asynchronous functions are language primitives, not library add-ons.
Self-hosting	The standard library MUST be written in pure Dryad and MUST use only the specified intrinsics.
Zero surprises	Every construct in this document has exactly one defined behaviour; undefined behaviour is not permitted.

Reading guide

A reader implementing an interpreter SHOULD read Clauses 1–15 in order. A reader implementing the bytecode VM SHOULD additionally read Clause 19. A reader implementing the AOT compiler SHOULD read Clauses 19 and 20. A reader implementing the standard library SHOULD read Clauses 8, 14, 17, and 18.

1 Scope

1.1 This document specifies the syntax, static semantics, and runtime semantics of the Dryad programming language, together with the normative requirements for its execution engines, native runtime interface, standard library, bytecode format, and ahead-of-time compilation pipeline.

1.2 This document specifies the observable behaviour of a conforming implementation. Observable behaviour comprises:

1. the result of evaluating every well-formed program;
2. the sequence of side effects produced during evaluation;
3. the errors and diagnostics reported for ill-formed programs;
4. the values returned by standard-library functions; and
5. the concurrency guarantees defined in Clause 15.

1.3 This document does not specify:

1. the hardware, operating system, or host environment on which an implementation runs, except where a boundary is explicitly defined;
2. the internal data structures, algorithms, or source-code organisation of an implementation;
3. user interfaces, editors, debuggers, package registries, or build tools, except where their behaviour is observable through a normative interface;
4. the contents of third-party libraries; or
5. performance, except where a normative requirement is explicitly stated.

1.4 A program is *well-formed* if it satisfies every early error rule in this document. A well-formed program has exactly one specified behaviour. This document does not define undefined behaviour; where a program violates a precondition, the implementation MUST report a runtime error as specified in Clause 14.

1.5 The unit of conformance is a *conforming implementation*, as defined in Clause 2. Conformance is claimed per conformance class, not per feature.

1.6 This document is organised as follows:

Clauses	Subject
1–6	Scope, conformance, references, terminology, notation, overview
7	Lexical grammar
8	Types and values
9	Expressions
10	Statements
11	Functions and classes
12	Modules
13	Execution model
14	Error system
15	Concurrency model
16	Foreign function interface
17	Runtime architecture and intrinsics
18	Standard library
19	Bytecode and intermediate representation
20	Ahead-of-time compilation
21–24	Paradigms, formal grammar, conformance testing, limitations
Annexes A–G	Registries, editorial decisions, proposals, bibliography

2 Conformance

2.1 A *conforming implementation* is a program that implements every normative requirement of this document for at least one conformance class, and that does not contradict any other normative requirement.

2.2 This document defines four conformance classes.

Class	Identifier	Required clauses	Description
Core	`core-1.0`	7–18	Lexical analysis, parsing, static semantics, interpretation, modules, errors, concurrency, FFI, runtime interface, standard library.
Virtual Machine	`vm-1.0`	7–19	Core, plus the bytecode compiler, bytecode verifier, and bytecode VM.
Ahead-of-Time	`aot-1.0`	7–20	Core, plus the bytecode pipeline and the AOT compiler.
Full	`full-1.0`	7–20	Core, VM, and AOT in a single implementation.

2.3 An implementation that claims a conformance class MUST:

1. accept every program that this document classifies as well-formed;
2. reject every program that this document classifies as ill-formed, with a diagnostic as specified in Clause 14;
3. produce the specified result for every well-formed program;
4. preserve the concurrency guarantees of Clause 15; and
5. satisfy every MUST or MUST NOT requirement in the clauses required by its class.

2.4 An implementation MAY provide extensions provided that all of the following hold:

1. the extension is not observable in a program that uses only the constructs specified in this document;
2. the extension does not alter the meaning of any specified construct;
3. the implementation documents the extension and the conditions under which it is active; and
4. disabling the extension restores a conforming implementation.

An implementation MUST NOT use the availability of an extension to justify a deviation from a normative requirement.

2.5 Certain behaviours are designated *implementation-defined* or *implementation-dependent*. For implementation-defined behaviour, a conforming implementation MUST document the chosen behaviour. For implementation-dependent behaviour, the implementation may choose freely and is not required to document the choice. This document identifies such behaviours explicitly; no other behaviour is implementation-defined or implementation-dependent.

2.6 A conforming implementation SHOULD pass the conformance test suite maintained by the Dryad Language Project for its claimed class. Passing the test suite is evidence of conformance but is not a substitute for the normative requirements of this document.

2.7 An implementation MUST report its edition and conformance class when invoked with `--version` or an equivalent mechanism. The report MUST identify this document as `DRYAD-SPEC-1.0-DRAFT`.

2.8 Conformance is not affected by:

1. the execution tier used to evaluate a program, provided the observable behaviour is preserved;
2. the host operating system, provided the specified platform abstractions are implemented;
3. the presence or absence of optional optimisations; or
4. the textual form of a diagnostic, provided it contains the information required by Clause 14.

2.9 An implementation that cannot support a required facility on a given host MUST reject programs that require that facility at the earliest specified point, with an error of category *unsupported* (Clause 14). Silent degradation is not conforming.

3 Normative References

3.1 The following documents are indispensable for the application of this specification. For dated references, only the cited edition applies. For undated references, the latest edition applies.

Reference	Title	Use in this document
RFC 2119	Key words for use in RFCs to Indicate Requirement Levels	Normative keywords (Clause 5.1)
RFC 8174	Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words	Normative keyword interpretation
RFC 3629	UTF-8, a transformation format of ISO 10646	Source-text encoding (Clause 7.1)
ISO/IEC 10646	Information technology — Universal Coded Character Set (UCS)	Character repertoire
The Unicode Standard	Version 15.1 or later	Character properties, identifier classes
IEEE 754-2019	IEEE Standard for Floating-Point Arithmetic	Number semantics (Clause 8.2.3)
ISO/IEC 9899:2018	Programming languages — C	Foreign function interface ABI (Clause 16)
System V ABI	System V Application Binary Interface, x86-64 and ARM64 editions	Calling conventions (Clauses 16, 20)
ELF gABI	System V gABI, Object File Format	AOT object output (Clause 20)
PE/COFF	Microsoft Portable Executable and Common Object File Format	AOT object output (Clause 20)
ECMA-262	ECMAScript Language Specification	Informative reference for algorithm style and grammar notation

3.2 The following documents are informative references. They describe the design context of this specification but do not create conformance requirements.

Reference	Title
DRYAD-TF-2.0	Dryad Theoretical Foundation, version 2.0
DRYAD-CONV-1.0	Dryad Conventions, version 1.0 (superseded)
LLVM Language Reference Manual	LLVM IR used by the AOT pipeline
OpenSSL documentation	TLS backend named by the standard library
POSIX.1-2017	Portable Operating System Interface

3.3 Where a normative reference is updated, the Dryad Language Project MAY publish an erratum adopting the update. Until such an erratum is published, the dated edition cited in Clause 3.1 applies.

4 Terms and Definitions

For the purposes of this document, the following terms and definitions apply. Terms are listed alphabetically. Cross-references use clause numbers.

Abstract operation. A named algorithm defined by this document. Abstract operations are not callable from Dryad source text; they define semantics.

Agent. An independent unit of execution with its own execution context and job queue. A Dryad program executes in at least one agent. Threads created by `thread function` execute in separate agents (Clause 15).

AOT compilation. Ahead-of-time compilation: translation of a Dryad program to native code before execution, as specified in Clause 20.

Binding. The association between an identifier and a value within an environment record.

Buffer. A contiguous, bounds-checked byte region with an explicit length and capacity (Clause 17.5).

Bytecode. The stack-based instruction format defined in Clause 19.

Call frame. The runtime record that stores the state of an active function invocation.

Class. A value that defines a nominal object type, its constructor, its instance members, and its static members (Clause 11).

Completion record. A specification value that records how evaluation of a statement or expression terminated. A completion record is one of *normal*, *break*, *continue*, *return*, or *throw* (Clause 10.2).

Conforming implementation. An implementation that satisfies Clause 2.

Directive. A syntactic construct beginning with `#` that instructs the implementation, such as a native-module directive `#<io>` (Clause 7.10), or a string-literal directive such as `"use strict types"` (Clause 8.6).

Dynamic type. The type of a value at runtime. Every value has exactly one dynamic type (Clause 8).

Early error. A condition that makes a program ill-formed and that MUST be detected before execution, during parsing or static analysis (Clause 14.8).

Environment record. A specification data structure that maps identifiers to bindings and that may have an outer environment reference (Clause 13.2).

Execution context. The specification state associated with the evaluation of a program, module, function, or block. An execution context includes a lexical environment, a variable environment, and a `this` binding (Clause 13.2).

Export. A binding made available by a module to importing modules (Clause 12.3).

Expression. A syntactic construct that evaluates to a value (Clause 9).

FFI. Foreign function interface: the boundary between Dryad and code implemented in another language (Clause 16).

Function. A callable value with a parameter list, a body, and a captured lexical environment (Clause 11).

HAL. Hardware abstraction layer: the small set of platform-specific operations beneath the intrinsic layer (Clause 17.4).

Host. The environment in which a conforming implementation executes, including its operating system, filesystem, network stack, and clock.

Identifier. A lexical token that names a binding, property, label, or module (Clause 7.5).

Implementation-defined. A behaviour for which a conforming implementation MUST document its choice (Clause 2.5).

Implementation-dependent. A behaviour for which a conforming implementation may choose freely without documentation (Clause 2.5).

Intrinsic. A primitive operation exposed by the runtime and declared in Dryad with `@intrinsic` (Clause 17.2).

Job. A unit of work scheduled on an agent's job queue, used to implement promises and asynchronous functions (Clause 15.4).

Lexical grammar. The grammar that defines how source text is divided into tokens (Clause 7).

Lexical token. An indivisible lexical unit produced by the lexer (Clause 7.4).

Module. A source-text unit with its own scope, exports, and module record (Clause 12).

Module record. The specification data structure that records a module's exports, imports, and evaluation state (Clause 12.1).

Module specifier. The string or identifier that names a module in an import declaration (Clause 12.4).

Native directive. The token `#<identifier>` that loads a native module (Clause 7.10).

Native module. A host-implemented module exposed through the FFI or the intrinsic layer (Clauses 16, 17).

Native type. A machine-level type used by the runtime, AOT compiler, and FFI, such as `f64`, `i32`, or `usize` (Clause 8.3).

Normal completion. A completion record indicating that evaluation continued to the next construct without an abrupt transfer of control (Clause 10.2).

Object. A value that groups properties and has an identity independent of its contents (Clause 8.3).

Observable behaviour. The behaviour listed in Clause 1.2.

Program. A collection of one or more modules, one of which is the entry module. Execution begins at the entry module (Clause 13.7).

Promise. A value representing the eventual result of an asynchronous operation (Clause 15.3).

Property. A named or indexed member of an object.

Realm. The set of intrinsic objects and global bindings associated with an agent. A realm is created when an agent is created (Clause 13.1).

Scope. The region of source text in which a binding is visible, together with the rules that determine

visibility (Clause 13.3).

Shape. A compiler-internal description of an object's property layout, used by the AOT compiler (Clause 20.4).

Source text. A sequence of Unicode code points encoded in UTF-8 (Clause 7.1).

Standard library (STDLIB). The collection of modules specified in Clause 18 and implemented in pure Dryad.

Statement. A syntactic construct that performs an action and produces a completion record rather than a value (Clause 10).

Static semantics. Rules that constrain well-formed programs and that are enforced before execution, including early errors and type checking (Clauses 8, 22).

Strict mode. The optional static typing mode enabled by `"use strict types"` (Clause 8.6).

Syscall. A primitive operation in the intrinsic set that corresponds to an operating-system service (Clause 17.3).

Thread. An independent agent created by a `thread function` declaration or a thread-creation operation (Clause 15.1).

Type. A set of values. The dynamic type of a value determines the operations that are defined on it (Clause 8).

Type annotation. A syntactic type expression attached to a declaration, parameter, return position, or expression (Clause 8.4).

Value. A member of a Dryad type. Values are the results of expressions and the contents of bindings (Clause 8).

Variable. A binding whose value may be changed after declaration.

Well-formed program. A program that satisfies every early error rule of this document (Clause 1.4).

5 Notational Conventions

5.1 Normative keywords

5.1.1 The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, NOT RECOMMENDED, MAY, and OPTIONAL in this document are to be interpreted as described in RFC 2119 and RFC 8174.

5.1.2 These keywords are normative only when they appear in uppercase. Lowercase uses of the same words are descriptive.

5.1.3 A requirement stated without a keyword is normative if it appears in a normative clause and describes observable behaviour, a well-formedness condition, or a conformance requirement.

5.1.4 The word *can* describes possibility; the word *cannot* describes impossibility under this specification. Neither word creates an optional implementation choice.

5.2 Grammar notation

5.2.1 Grammars in this document use an extended Backus–Naur form (EBNF) with the following metacharacters.

Notation	Meaning
`::=`	Production definition.
`<name>`	Nonterminal symbol `name`.
`"text"`	Terminal symbol consisting of the exact characters `text`.
`A | B`	Alternative: either `A` or `B`.
`[ A ]`	Optional occurrence of `A` (zero or one).
`{ A }`	Repetition of `A` (zero or more).
`( A )`	Grouping.
`ε`	The empty production.
`(* text *)`	Informative comment inside a grammar.

5.2.2 A production of the form `<A> ::= <B> { "," <B> }` denotes a possibly empty comma-separated sequence of `<B>`. A trailing comma is permitted only where a production says so explicitly.

5.2.3 Lexical grammar productions are identified by the prefix `Lexical` and are defined in Clause 7. Syntactic grammar productions are defined in Clause 22 and referenced throughout this document. When a syntactic production references a token, the token name is written in angle brackets, for example `<Identifier>`.

5.2.4 The lexical grammar is applied before the syntactic grammar. The lexer consumes the longest sequence of input characters that forms a valid token, subject to the rules of Clause 7.11. Whitespace and comments separate tokens but are not tokens themselves.

5.2.5 A syntactic grammar is unambiguous if every well-formed token sequence has exactly one parse tree. This document defines an unambiguous grammar. An implementation MUST reject a token sequence for which no parse tree exists and MUST report an early error if more than one parse tree exists.

5.2.6 Grammar productions do not by themselves impose evaluation order. Evaluation order is defined by the algorithms in Clauses 9 through 13.

5.3 Algorithm conventions

5.3.1 Semantic algorithms are written as numbered steps. Steps are performed in order unless a step transfers control.

5.3.2 The phrase *Let x be …* introduces a local specification variable `x`. Specification variables are distinct from Dryad variables and are not observable by a program.

5.3.3 The phrase *If condition, then …* executes the indented steps when `condition` is true. *Otherwise* executes them when `condition` is false. *If condition, then …; otherwise …* is equivalent.

5.3.4 The phrase *Return value* terminates the abstract operation and yields `value`. The phrase *Return* without a value yields the normal completion `Normal(empty)`.

5.3.5 The phrase *Assert: condition* states an invariant that MUST hold whenever the algorithm reaches that step. A violation indicates an error in this specification or in a conforming implementation, not in the source program.

5.3.6 An abstract operation whose name ends in `?` MAY return an abrupt completion. In an algorithm, the notation `result = Operation?` means: perform `Operation`; if it returns an abrupt completion, return that completion immediately; otherwise, assign its value to `result`.

5.3.7 Abstract operations named in CamelCase, such as `GetValue` or `ResolveBinding`, are specification operations. They are not callable from Dryad source text.

5.3.8 An algorithm MAY state that behaviour is *implementation-defined* or *implementation-dependent*. Those terms have the meanings given in Clause 2.5.

5.4 Completion records

5.4.1 A completion record is a specification value of one of the following forms.

Form	Payload	Meaning
`Normal(value)`	A value or `empty`	Evaluation continued normally.
`Break(label)`	A label or `empty`	Control transfers to the matching labelled statement.
`Continue(label)`	A label or `empty`	Control transfers to the next iteration of the matching loop.
`Return(value)`	A value	Control returns from the current function.
`Throw(value)`	A value	An exception is propagating.

5.4.2 A completion record is *abrupt* if it is not `Normal`. A completion record is *empty* if it carries no value.

5.4.3 Statement evaluation in Clause 10 yields completion records. Expression evaluation in Clause 9 yields values or throws; an expression never yields `Break`, `Continue`, or `Return`.

5.5 Type notation

5.5.1 This document uses the following notation for value types.

Notation	Meaning
`Null`	The type whose only value is `null`.
`Boolean`	`true` and `false`.
`Number`	IEEE 754 binary64 values, including `NaN` and infinities.
`String`	Finite sequences of Unicode scalar values.
`Object`	Objects with identity, including arrays, tuples, functions, and class instances.
`Array<T>`	An ordered, dynamically sized sequence of `T`.
`Tuple<T1, ..., Tn>`	A fixed-length sequence with element types `T1` through `Tn`.
`Function<P..., R>`	A callable value with parameters `P...` and result `R`.
`Class<C>`	The type of instances of class `C`.
`Buffer`	The byte-buffer type of Clause 17.5.
`Promise<T>`	The asynchronous-result type of Clause 15.3.
`Result<T, E>`	A value that is either `Ok(T)` or `Err(E)` (Clause 8.3.7).
`Thread<T>`	A handle to a thread producing `T` (Clause 15.1).
`Mutex`	A mutual-exclusion primitive (Clause 15.2).
`any`	The union of all value types.

Notation	Meaning
`never`	The empty type; an expression of type `never` does not return normally.
`T?`	Shorthand for `T | Null`.
`T | U`	The union of `T` and `U`.

5.5.2 This document uses the following notation for native types, which are visible to the AOT compiler, the FFI, and intrinsic declarations.

Notation	Meaning
`f64`	IEEE 754 binary64.
`i8`, `i16`, `i32`, `i64`	Signed two's-complement integers.
`u8`, `u16`, `u32`, `u64`	Unsigned integers.
`usize`, `isize`	Pointer-sized unsigned and signed integers.
`bool`	A native boolean.
`ptr<T>`	A raw pointer to `T`.
`void`	No value.

5.5.3 `T <: U` means that `T` is a subtype of `U`; every value of type `T` is also a value of type `U`. `T ~ U` means that `T` and `U` are consistent: their intersection is non-empty and values may flow between them under the rules of Clause 8.5.

5.5.4 `typeof(v)` denotes the dynamic type of value `v`. `IsCallable(v)` denotes whether `v` is a function or class constructor. `IsObject(v)` denotes whether `v` is an object.

5.6 Mathematical notation

5.6.1 `R` denotes the real numbers, `Z` the integers, and `B` the booleans.

5.6.2 A sequence is written `<a1, ..., an>`; an empty sequence is `<>`. The length of sequence `s` is `len(s)`. Indexing is zero-based and written `s[i]`.

5.6.3 A map is written `{k1 -> v1, ..., kn -> vn}`; `m[k]` denotes lookup and `m[k -> v]` denotes extension or update.

5.6.4 The interval `[a, b)` denotes the set of integers `x` such that `a ≤ x < b`. `[a, b]` includes `b`.

5.6.5 `floor(x)` denotes the largest integer less than or equal to `x`; `trunc(x)` denotes the integer part of `x` toward zero.

5.6.6 `SameValue(x, y)` denotes the specification equality relation of Clause 8.9. `SameValueZero(x, y)` is `SameValue` except that it treats `+0` and `-0` as equal.

5.7 Normative and informative marking

5.7.1 A clause is normative unless its heading or introductory paragraph marks it informative. Annexes A through D are normative; Annexes E through G are informative.

5.7.2 Within a normative clause, a paragraph beginning with **NOTE** is informative and does not create a requirement.

5.7.3 Examples are informative unless the example is introduced as a normative test vector.

5.8 Document versioning

5.8.1 This document is Edition 1.0, Draft. Clause numbers are stable within the edition. An erratum MUST NOT renumber a clause; it MAY add subclauses with a new final component.

5.8.2 A future edition MAY renumber clauses only if it publishes a migration table mapping old clause numbers to new ones.

6 Overview

6.1 Language identity

6.1.1 Dryad is a statically scoped, dynamically typed, multi-paradigm programming language. It supports procedural, object-oriented, and functional programming in a single surface syntax.

6.1.2 Dryad source text is compiled to an internal program representation and may be executed by three interchangeable tiers: a tree-walking interpreter, a bytecode virtual machine, or an ahead-of-time compiled native binary.

6.1.3 The language provides first-class functions, closures, classes with single inheritance, interfaces, modules, exceptions, asynchronous functions, operating-system threads, mutexes, a foreign function interface, and a standard library written entirely in Dryad.

6.2 Design axioms

This subclause is informative. The axioms guide interpretation where this document leaves room for judgement; they do not override an explicit requirement.

Axiom 1 — Clarity over performance. Readability and predictable behaviour take precedence over premature optimisation. An optimisation is conforming only if it preserves observable behaviour.

Axiom 2 — Syntactic consistency. Syntactic constructs follow uniform, predictable patterns. Similar operations use similar notation.

Axiom 3 — Informative errors. Every error reports a category, a numeric code, a source location, and, where the cause is determinable, a suggested correction (Clause 14).

Axiom 4 — Composability. Language primitives combine without special cases. A value of a given type may be used wherever that type is accepted.

Axiom 5 — Zero surprises. Every construct has exactly one specified behaviour. The implementation MUST NOT depend on unspecified state, ambient configuration, or evaluation order that this document leaves undefined.

6.3 Surface language

6.3.1 A minimal Dryad program is a sequence of statements. The following program is well-formed and prints `42`:

```
function answer() {
    return 42;
}

print(answer());
```

6.3.2 The principal syntactic categories are:

Category	Examples	Clause
Declarations	`let`, `const`, `function`, `class`, `import`, `export`	10–12
Statements	`if`, `while`, `for`, `try`, `return`, `throw`	10
Expressions	operators, calls, member access, `match`, arrow functions	9
Types	primitives, arrays, tuples, functions, classes, native types	8
Modules	named, namespace, side-effect, and identifier imports	12
Directives	`#<io>`, `"use strict types"`, `@compile(...)`	7, 8, 13

6.4 Execution architecture

6.4.1 A conforming implementation MUST provide at least one execution tier and SHOULD provide all three. The tiers are:

Tier	Input	Requirement
Interpreter	Abstract syntax tree	Evaluate the tree directly.
Bytecode VM	Bytecode module (Clause 19)	Execute verified bytecode.
AOT	Native object code (Clause 20)	Execute ahead-of-time compiled code.

6.4.2 All tiers MUST be observationally equivalent for every well-formed program. Tier selection is not observable except through the timing and the diagnostic facilities explicitly defined in Clause 20.

6.4.3 The interpreter MUST collect the runtime type observations required by the metadata header of Clause 19.8 when the implementation also provides a bytecode or AOT tier. An implementation that provides only the interpreter is not required to emit metadata.

6.4.4 The `@compile` decorator (Clause 13.9) selects the execution tier for a function or block. In the absence of a decorator, the tier is selected by the implementation or by the implementation's command-line interface.

6.5 Native boundary and self-hosting

6.5.1 A Dryad implementation consists of a native runtime and a standard library. The native runtime MUST expose only the intrinsics specified in Clause 17.3 and Annex C. The standard library MUST be implemented in pure Dryad and MUST obtain all host services through those intrinsics.

6.5.2 An implementation MAY provide additional host facilities through the foreign function interface of Clause 16. Such facilities are not part of the standard library and do not extend the intrinsic set.

6.5.3 The native runtime MUST NOT contain language-specific standard library logic beyond the intrinsics. In particular, collection types, formatting, HTTP, JSON, cryptography, and path handling are implemented in Dryad.

6.6 Document organisation

6.6.1 The normative clauses are organised by language layer. Clauses 7–12 define the source language: lexical structure, values and types, expressions, statements, declarations, and modules. Clause 13 defines how those constructs execute. Clause 14 defines errors. Clause 15 defines concurrency. Clauses 16 and 17 define the native boundary. Clause 18 defines the library. Clauses 19 and 20 define the compiled representations.

6.6.2 Clause 22 contains the complete grammar. A production in Clause 22 is normative. Syntax shown in other clauses is explanatory and MUST agree with Clause 22; if it does not, Clause 22 prevails and the discrepancy is an erratum.

6.6.3 Annex A is the error-code registry. Annex B is the opcode registry. Annex C is the intrinsic registry. Annex D is the standard-library index. Annex E records editorial decisions against the conventions draft. Annex F is the informative proposals annex. Annex G is the bibliography.